

Library Management System

React + MockAPI — Student Explainer

This guide explains a simple Library Management System with two roles: Librarian and User. The Librarian can create, update, delete, issue and cancel books. The User can view books and their availability.

1. Overall Flow

React Frontend → MockAPI REST API → Book data

React keeps displayed books in state. Changes are sent to MockAPI, then GET is called again to refresh the screen.

Note: MockAPI is a hosted mock REST API. It is not the same as SQL Server database connectivity.

2. Book Object

Each book should contain: **id**, **title**, **author**, and **isIssued**.

```
{"id":"8","title":"Harry Potter","author":"J.K. Rowling","isIssued":false}
```

3. API Base URL

```
const API = "https://6aab8f45ea0e22daa6dc5148.mockapi.io/Books";  
  
In this u shld add ur own URL!
```

4. GET — Display Books

```
const getBooks = async () => { const response = await fetch(API); const data = await  
response.json(); setBooks(data); };
```

GET retrieves all books and stores them in React state.

5. POST — Create Book

```
const addBook = async (book) => { await fetch(API, { method: "POST", headers: {  
"Content-Type": "application/json" }, body: JSON.stringify(book) }); getBooks(); };
```

New books should start with **isIssued: false**.

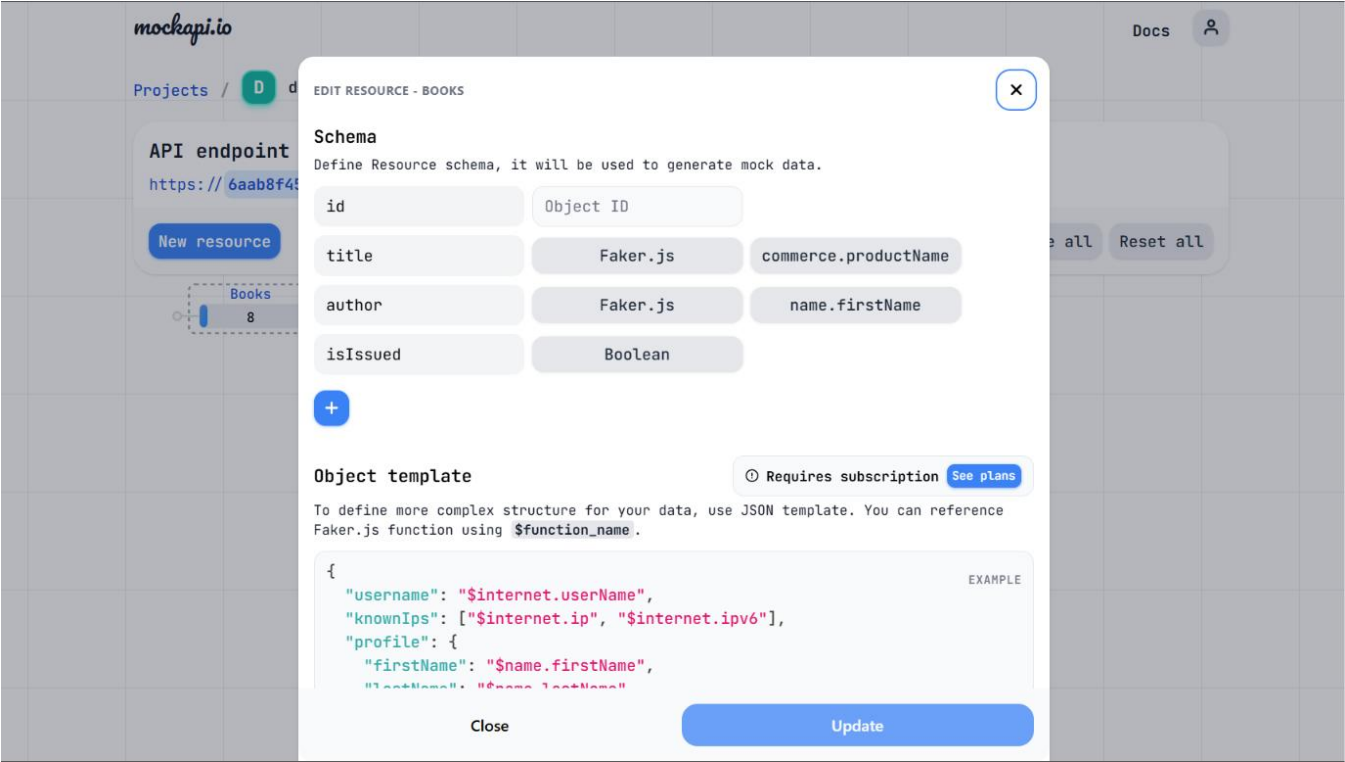
6. PUT — Update Book

```
await fetch(`${API}/${id}`, { method: "PUT", headers: { "Content-Type":  
"application/json" }, body: JSON.stringify(book) });
```

7. DELETE — Delete Book

```
await fetch(`${API}/${id}`, { method: "DELETE" });
```

Mock Api should be generated of Book object



8. Correct Issue Operation

```
const issueBook = async (id) => { await fetch(`${API}/${id}`, { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ isIssued: true }) });  
getBooks(); };
```

The important change is from **/Books/8/issue** to **/Books/8**. The book's `isIssued` field becomes true.

9. Correct Cancel / Return Operation

```
const cancelIssue = async (id) => { await fetch(`${API}/${id}`, { method: "PUT", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ isIssued: false }) });  
getBooks(); };
```

Changing `isIssued` to false makes the book available again.

10. Librarian vs User

Operation	Librarian	User
View books	Yes	Yes
Create book	Yes	No
Update book	Yes	No
Delete book	Yes	No
Issue book	Yes	No
Cancel/Return	Yes	No
See status	Yes	Yes

11. Conditional Rendering

```
{user.role === "Librarian" && ( issueBook(book.id)>> Issue )}
```

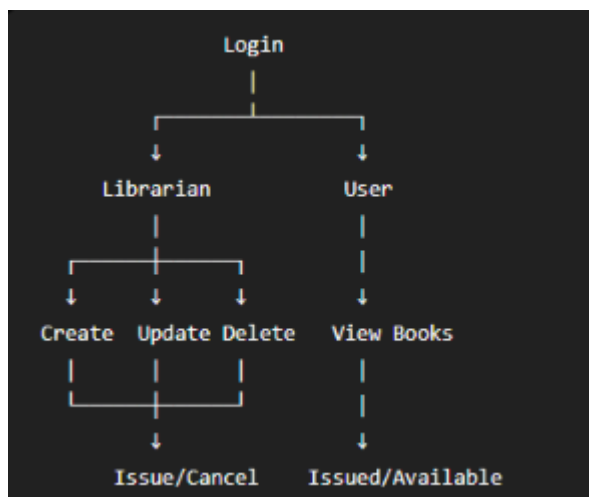
This displays Librarian controls only when the selected role is Librarian.

12. Display Issued / Available

```
{book.isIssued ? ( Issued ) : ( Available )}
```

This uses the JavaScript ternary operator inside JSX.

13. Complete Issue Flow



1. Librarian clicks Issue.
2. React calls PUT /Books/8.
3. Body contains { isIssued: true }.
4. MockAPI updates book 8.
5. React calls GET /Books again.
6. UI displays Issued.

14. Concepts Students Learn

- useState — state and form values.
- useEffect — loading data.
- Props — passing data/functions between components.
- fetch() — API communication.

- GET, POST, PUT, DELETE — REST operations.
- JSON.stringify() — request JSON.
- Conditional rendering — role-based UI.
- map() — displaying books.
- Ternary operator — Issued/Available status.

15. Important Limitation

This is intentionally simple for a FrontEnd Web Technology assignment. The role selector is frontend-only, so it is not real authentication/authorization. MockAPI is also not SQL Server. If SQL Server is required, use an ASP.NET Core Web API connected to SQL Server.

16. Final Architecture

Login / Role → React Components → fetch() → MockAPI → Book Data → React State → UI

The API stores the book records, React displays them, and Librarian actions modify the book record. There is no local array acting as the permanent data source.

Code:

```
import { useEffect, useState } from "react";
import Login from "../libManagement/Login";
import BookForm from "../libManagement/BookForm";
import BookList from "../libManagement/BookList";
import "../libManagement/lib.css";

const API = "https://6aab8f45ea0e22daa6dc5148.mockapi.io/Books";

function App() {
  const [user, setUser] = useState(null);
  const [books, setBooks] = useState([]);

  // GET ALL BOOKS
  const getBooks = async () => {
    // try {
    //   const res = await fetch(API);
    //   const data = await res.json();

    //   setBooks(data);
    // } catch (error) {
    //   console.log(error);
    // }

    fetch(API)
      .then((res) => res.json())
      .then((data) => setBooks(data))
      .catch((error) => console.log(error));
  };

  // ADD BOOK
  const addBook = async (book) => {
    try {
      await fetch(API, {
        method: "POST",
        headers: {
```

```

        "Content-Type": "application/json",
    },
    body: JSON.stringify(book),
  });

  getBooks();
} catch (error) {
  console.log(error);
}
};

// UPDATE BOOK
const updateBook = async (id, book) => {
  try {
    await fetch(`${API}/${id}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(book),
    });

    getBooks();
  } catch (error) {
    console.log(error);
  }
};

// DELETE BOOK
const deleteBook = async (id) => {
  try {
    await fetch(`${API}/${id}`, {
      method: "DELETE",
    });

    getBooks();
  } catch (error) {
    console.log(error);
  }
};

// ISSUE BOOK
const issueBook = async (id) => {
  try {
    await fetch(`${API}/${id}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        isIssued: true
      })
    });
  }
};

```

```

        getBooks();
    } catch (error) {
        console.log(error);
    }
};

// CANCEL / RETURN BOOK
const cancelIssue = async (id) => {
    try {
        await fetch(`${API}/${id}`, {
            method: "PUT",
            headers: {
                "Content-Type": "application/json"
            },
            body: JSON.stringify({
                isIssued: false
            })
        });

        getBooks();
    } catch (error) {
        console.log(error);
    }
};

```

```

// GET BOOKS AFTER LOGIN

```

```

useEffect(() => {
    if (user) {
        getBooks();
    }
}, [user]);

```

```

// LOGIN PAGE

```

```

if (!user) {
    return <Login setUser={setUser} />;
}

```

```

return (
    <div className="container">

        <div className="header">
            <h1>Library Management System</h1>

            <div>
                <span>
                    Logged in as: <b>{user.role}</b>
                </span>

                <button
                    className="logout"
                    onClick={() => setUser(null)}
                >
                    Logout
                </button>
            </div>

```

```

        </div>
    </div>

    { /* ONLY LIBRARIAN CAN ADD BOOK */ }
    { user.role === "Librarian" && (
        <BookForm addBook={addBook} />
    ) }

    <BookList
        books={books}
        user={user}
        updateBook={updateBook}
        deleteBook={deleteBook}
        issueBook={issueBook}
        cancelIssue={cancelIssue}
    />

</div>
);
}

export default App;

```

```

import { useState } from "react";

function BookForm({ addBook }) {

    const [title, setTitle] = useState("");
    const [author, setAuthor] = useState("");

    const handleSubmit = (e) => {
        e.preventDefault();

        if (title === "" || author === "") {
            alert("Please enter all details");
            return;
        }

        addBook({
            title: title,
            author: author
        });

        setTitle("");
        setAuthor("");
    };

    return (
        <div className="form-box">

            <h2>Add Book</h2>

```

```

    <form onSubmit={handleSubmit}>

      <input
        type="text"
        placeholder="Book Title"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
      />

      <input
        type="text"
        placeholder="Author"
        value={author}
        onChange={(e) => setAuthor(e.target.value)}
      />

      <button type="submit">
        Add Book
      </button>

    </form>

  </div>
);
}

export default BookForm;

```

```

import { useState } from "react";

function BookList({
  books,
  user,
  updateBook,
  deleteBook,
  issueBook,
  cancelIssue
}) {

  const [editId, setEditId] = useState(null);
  const [editTitle, setEditTitle] = useState("");
  const [editAuthor, setEditAuthor] = useState("");

  // START EDIT
  const startEdit = (book) => {
    setEditId(book.id);
    setEditTitle(book.title);
    setEditAuthor(book.author);
  };

  // SAVE EDIT
  const saveEdit = (id) => {

```

```

updateBook(id, {
  title: editTitle,
  author: editAuthor
});

setEditId(null);
};

return (
  <div>

    <h2>Book List</h2>

    <table>

      <thead>
        <tr>
          <th>ID</th>
          <th>Title</th>
          <th>Author</th>
          <th>Status</th>

          { /* Only Librarian sees Action */ }
          {user.role === "Librarian" && (
            <th>Action</th>
          )}
        </tr>
      </thead>

      <tbody>

        {books.map((book) => (

          <tr key={book.id}>

            <td>{book.id}</td>

            <td>
              {editId === book.id ? (
                <input
                  value={editTitle}
                  onChange={(e) =>
                    setEditTitle(e.target.value)
                  }
                />
              ) : (
                book.title
              )}
            </td>

            <td>
              {editId === book.id ? (
                <input

```

```

        value={editAuthor}
        onChange={(e) =>
            setEditAuthor(e.target.value)
        }
    />
) : (
    book.author
)}
</td>

<td>

    {book.isIssued ? (
        <span className="issued">
            Issued
        </span>
    ) : (
        <span className="available">
            Available
        </span>
    )}

</td>

{/* LIBRARIAN ACTIONS */}
{user.role === "Librarian" && (

    <td>

        {editId === book.id ? (

            <button
                onClick={() => saveEdit(book.id)}
            >
                Save
            </button>

        ) : (

            <button
                onClick={() => startEdit(book)}
            >
                Edit
            </button>

        )}

        <button
            onClick={() => deleteBook(book.id)}
        >
            Delete
        </button>

        {!book.isIssued ? (

```

```

        <button
          onClick={() => issueBook(book.id)}
        >
          Issue
        </button>

      ) : (

        <button
          onClick={() => cancelIssue(book.id)}
        >
          Cancel
        </button>

      )}

    </td>

  )}

</tr>

)}}

</tbody>

</table>

</div>
);
}

export default BookList;

```

```

import { useState } from "react";

function Login({ setUser }) {

  const [role, setRole] = useState("");

  const handleLogin = (e) => {
    e.preventDefault();

    if (role === "") {
      alert("Please select a role");
      return;
    }

    setUser({
      role: role
    });
  };
}

```

```
return (  
  <div className="login">  
  
    <h1>Library Management System</h1>  
  
    <form onSubmit={handleLogin}>  
  
      <h2>Login</h2>  
  
      <select  
        value={role}  
        onChange={(e) => setRole(e.target.value)}  
      >  
        <option value="">Select Role</option>  
        <option value="Librarian">Librarian</option>  
        <option value="User">User</option>  
      </select>  
  
      <button type="submit">  
        Login  
      </button>  
  
    </form>  
  
  </div>  
>);  
}  
  
export default Login;
```